Reporte

Nombre: Jaime Sebastian Esparza Leyva Alan Joel Sánchez García Jorge Alejandro Jiménez González	**Matrícula**: 2097304 2082464 2097849
Nombre del curso: Componente de lógica de negocios	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: NA	**Actividad**: Architecture Decision Records (ADR)
Fecha: 15/10/2026	
Bibliografia (Formato APA) NA	

Architecture Decision Records (ADR)

Se documentan cuatro decisiones con consecuencias para el diseño. Todas están en estado Propuesta hasta que el equipo las revise y las acepte.

ADR-01: Domain Model como patrón principal

Campo	Contenido
Estado	Propuesta
Contexto	Sazón Urbano necesita proteger reglas de disponibilidad, estados del pedido, trazabilidad y promociones (RN-02 a RN-13). El BRD describe el problema como falta de sincronía entre pedidos, cocina, ventas e inventario, y exige considerar el estado del pedido al modificar o cancelar.
Alternativas	1) Transaction Script: simple, pero duplica reglas entre casos de uso. 2) Table Module: ordena por tabla, pero las reglas que cruzan conceptos quedan sin dueño. 3) Domain Model con consultas en Transaction Script (elegida).
Decisión	Domain Model para todo lo que cambia estado y Transaction Script solo para las consultas de lectura.
Consecuencias	**Beneficios:** reglas e invariantes en un solo lugar; trazabilidad por construcción. **Costos:** más código y curva de aprendizaje. **Riesgos:** sobrediseño si se modelan conceptos innecesarios. **Trabajo posterior:** validar con el equipo los supuestos SP-01 a SP-09.
Trazabilidad	N-03, N-05, N-06, N-08; RN-02, RN-06, RN-07, RN-08, RN-13; US-04, US-11, US-15; BPMN: ¿Producto disponible?, ¿Pedido completo?

ADR-02: Pedido como un único Aggregate con líneas, preparación e historial

Campo	Contenido
Estado	Propuesta
Contexto	El pedido pasa por varias estaciones y solo está completo cuando todas sus líneas están listas (RN-06). Cada cambio debe quedar trazado (RN-09).
Alternativas	1) Una línea de pedido como Aggregate propio. 2) Un Aggregate OrdenEstacion por estación. 3) El Pedido contiene líneas, estado de preparación e historial (elegida).
Decisión	Las líneas, su estado de preparación y el historial viven dentro del Aggregate Pedido.
Consecuencias	**Beneficios:** la regla «pedido completo» se garantiza en una sola transacción. **Costos y riesgos:** dos estaciones que marcan al mismo tiempo pueden causar un conflicto de versión; se resuelve reintentando. Es aceptable por el volumen de un restaurante. **Trabajo posterior:** medir la frecuencia de conflictos.
Trazabilidad	N-02, N-05; RN-05, RN-06, RN-09; US-07, US-08, US-12; BPMN: Reúne todas las partes, ¿Pedido completo?

ADR-03: Consumo de ingredientes al confirmar el pedido, en una sola transacción

Campo	Contenido
Estado	Propuesta
Contexto	El BRD reporta productos que se registran y luego resulta que no hay con qué prepararlos, y diferencias entre ventas e inventario. Hoy el inventario se revisa al final del día (BPMN).
Alternativas	1) Mantener el inventario solo al final del día (AS-IS): no resuelve el problema. 2) Descontar al cerrar el pedido: el producto puede venderse sin existencia durante la preparación. 3) Descontar al confirmar en la misma transacción, con concurrencia optimista (elegida). 4) Descontar mediante eventos con consistencia eventual: el pedido se confirmaría antes de asegurar la existencia.
Decisión	confirmarPedido cambia el Pedido y los Ingredientes de su receta en una sola transacción (SP-04), con control de versión por Aggregate.
Consecuencias	**Beneficios:** el inventario esperado se mantiene al día y no se confirma un producto sin existencia. **Costos y riesgos:** rompe de forma consciente la regla de un Aggregate por transacción; los ingredientes muy usados pueden generar conflictos de versión y reintentos. **Trabajo posterior:** validar SP-04 y SP-08 (reintegro al cancelar).
Trazabilidad	N-03, N-04; RN-02, RN-10, RN-12, RN-15; US-04, US-09, US-14; BPMN: Revisa existencias, Registra entradas y salidas, Inventario al final del día.

ADR-04: CQRS básico sin bases de datos separadas

Campo	Contenido
Estado	Propuesta
Contexto	El gerente y el encargado necesitan consultar ventas, historial y comparativos (US-12, US-13, US-14); la cocina necesita ver solo sus productos (US-07). Cargar Aggregates para esas lecturas sería ineficiente.
Alternativas	1) Sin separación: leer y escribir con el mismo modelo. 2) CQRS completo con bases de datos y modelos separados. 3) CQRS básico: Commands con el dominio y Queries con DTO de lectura sobre la misma base (elegida).
Decisión	Los métodos de escritura son Commands con Handler que usan el dominio; las consultas son Queries que devuelven DTO sin cargar Aggregates.
Consecuencias	**Beneficios:** consultas simples y responsabilidades claras. **Costos:** hay que mantener DTO y consultas aparte; VentaResumen se actualiza con el evento PedidoCerrado. **Riesgos:** si el evento no se procesa, la lectura de ventas queda desactualizada. **Trabajo posterior:** definir cómo reintentar la actualización del resumen.
Trazabilidad	N-02, N-05, N-07; RN-14, RN-15; US-07, US-12, US-13, US-14.